Logarithmic Merging in C++

Implementation and experimental evaluation | Information Retrieval programming assignment

Student: Ahmad Essawii **Student ID:** 12429664 **Date:** October 2026

1. Implementation

The implementation extends the instructor-provided **DynamicIndex** interface through **LogarithmicIndex**. Incoming documents are added to the in-memory auxiliary index Z0, which is limited by the number of distinct (term, document) postings. When a new document would exceed that capacity, the framework invokes flush() before adding that document. The provided infrastructure handles tokenization, index encoding, I/O counters and streaming disk merges; only src/logarithmic.cpp was implemented for this assignment.

Generation invariant. A vector of base paths, gen_, represents levels I0, I1, ...; an empty string denotes a vacant level. Each occupied level i combines 2^i flushed batches and holds at most 2^i n postings. The occupied levels correspond to set bits in a binary flush counter. Thus, after F overflow-triggered flushes and before finalization, generation_mask() equals F.

Flush procedure. A nonempty Z0 is written to a new temporary index and then cleared. The new index is treated as a carry. At the first unoccupied level it is stored; at an occupied level, merge_indexes({existing_generation, carry}, new_output) creates a merged carry, and both consumed inputs are removed after releasing their cached readers. The older generation is always passed first, preserving the chronological order of disjoint document ranges and enabling the streaming merge helper's append-friendly path. Each successful merge increments stats_.merges.

Queries. lookup(term) begins with the postings from Z0, then unions them with disk_lookup results from each occupied generation. The provided union_postings routine returns sorted, duplicate-free document IDs. Because the query covers both memory and every disk generation, it remains correct while indexing proceeds.

Finalization and cleanup. finalize(out_base) flushes remaining Z0 data. It writes an empty index if there are no generations, renames the only generation if there is exactly one, or performs one oldest-to-newest k-way merge if several levels are occupied. Cached readers are forgotten before rename/deletion, consumed levels are cleared, and the destructor deletes any remaining temporary generations. disk_indexes() counts nonempty levels; generation_mask() sets bit i only for occupied levels.

Complexity. With T total postings and an auxiliary capacity n, the number of flushes is approximately T/n. The auxiliary-plus-main baseline rewrites a growing main index after each flush, yielding roughly $O(T^2/n)$ postings written. Logarithmic merging rewrites a posting across at most O(log(T/n)) levels, giving O(T log(T/n)) merge work. Exact measurements differ because batches may be smaller than n and finalization adds writes.

2. Experiments and measurements

The instructor-provided 50,000-document synthetic corpus was used for all eight configurations (two policies times four auxiliary posting capacities). Indexes were compiled and executed using the Windows/MinGW setup. All eight final indexes passed verification against the reference: **50,000 terms match**. The user also reported 12/12 correctness tests passing on each of the tiny and large test collections.

E1 — Total index-write cost

n	Aux writes (M)	Log writes (M)	Aux / Log	Log max indexes
50,000	327.31	36.01	9.09x	6
100,000	165.00	30.38	5.43x	5
200,000	86.58	24.79	3.49x	4
400,000	47.19	19.20	2.46x	3

Table 1. Measured postings written **before finalization**, aggregated from results/dynamic.csv. The ratio describes reduction in indexing writes, not end-to-end speed.

Run-time measurements

n	Aux time (s)	Log time (s)	Aux write amp.	Log write amp.
50,000	27.64	17.19	57.64	6.34
100,000	16.12	12.21	29.06	5.35
200,000	10.18	8.44	15.25	4.37
400,000	6.60	5.89	8.31	3.38

Table 2. Elapsed indexing-plus-finalization seconds, and pre-finalization write amplification, from the same experimental run. These wall-clock times are machine- and workload-dependent.

Experimental environment

Platform: Windows PowerShell; **compiler:** MSYS2 g++ 14.2.0; **build:** CMake Release with MinGW Makefiles. **CPU:** 13th Gen Intel Core i7-13620H; **RAM:** 16 GB physical memory (16,788,709,376 bytes).

Verification

The eight verifier logs each report “OK: 50000 terms match the reference index.” This validates the finalized index content for all configurations, while the two reported 12/12 test runs also exercise dynamic lookup and generation bookkeeping. Correctness does not by itself establish a performance advantage, so I/O writes and runtime are compared separately.

3. Growth traces (E2)

For the required n = 100,000 setting, the following figures use the per-flush traces, not interpolated or copied measurements. Trace rows are recorded after overflow-triggered flushes; the finalization write is not included in the cumulative-write trace.

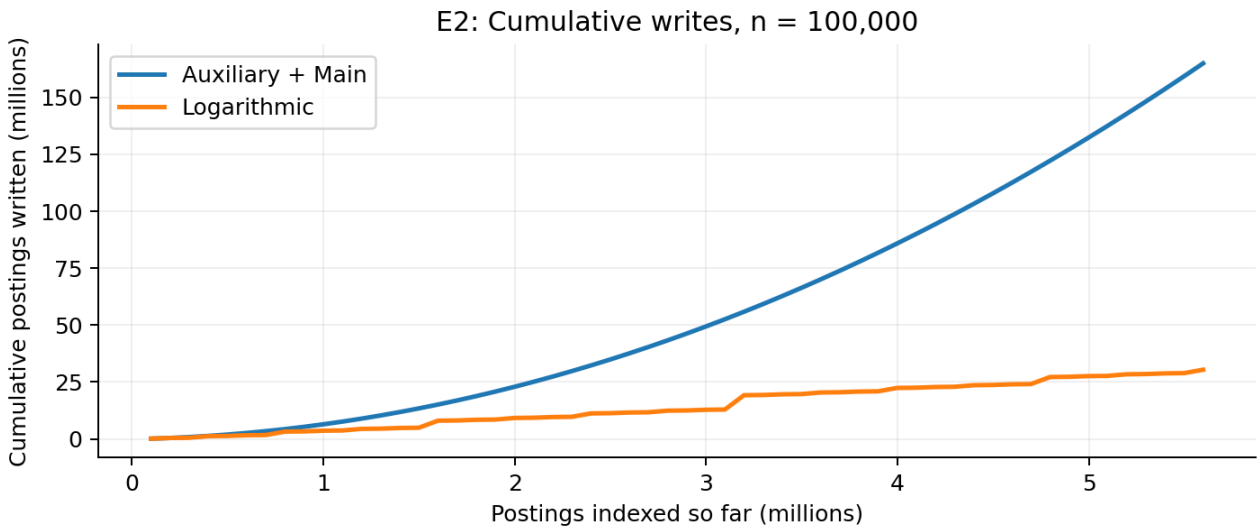


Figure 1. Cumulative postings written versus postings indexed so far for Auxiliary + Main and Logarithmic Merging. Both are drawn on the same axes.

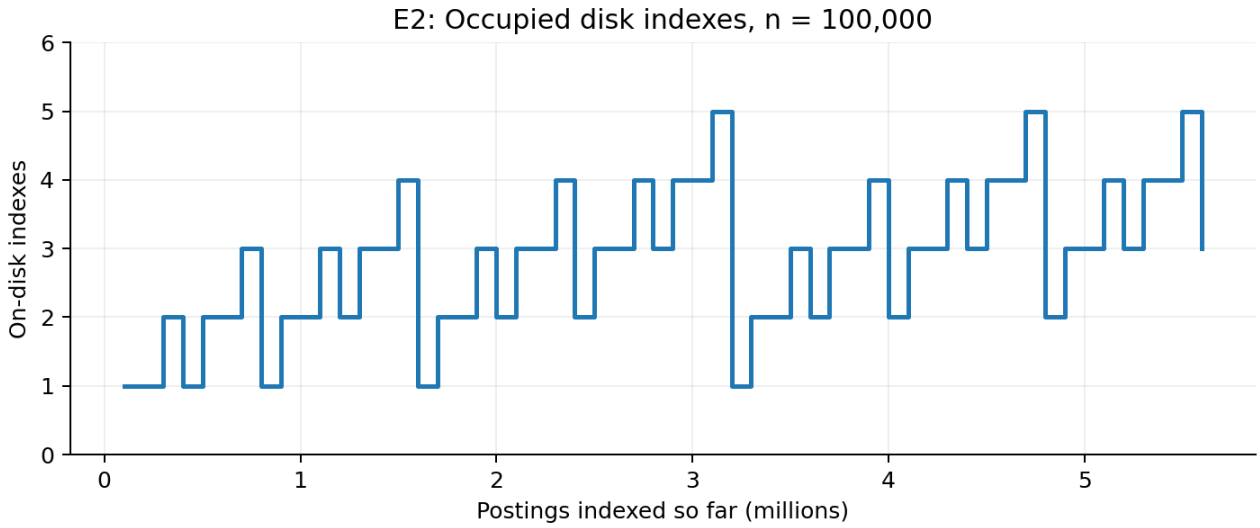


Figure 2. Number of occupied on-disk indexes versus postings indexed so far for Logarithmic Merging. The non-monotonic index count reflects binary carries combining occupied levels.

4. Interpretation and discussion

Write cost and observed improvements

At n = 100,000, the baseline wrote 165,003,237 postings during indexing, versus 30,381,950 for logarithmic merging, a 5.43x reduction. At n = 50,000 the reduction was 9.09x; it fell to 2.46x at n = 400,000. This trend is consistent with fewer flushes at higher capacities: the baseline rewrites its expanding main index fewer times, reducing the relative benefit of the logarithmic policy. The logarithmic approach nevertheless wrote fewer postings in all four measured configurations.

What the curves show

In Figure 1, the baseline cumulative-write curve becomes increasingly steep as the main index grows: each additional flush rewrites a larger amount of old content. The logarithmic curve grows in steps associated with carries; relatively inexpensive flushes alternate with larger multi-level merges. Figure 2 makes this behavior visible as the number of occupied disk levels rises and collapses according to the binary representation of the flush count.

Finalization and alternative merging strategy

The finalization step adds one more write of the complete index when multiple generations remain. For n = 100,000, logarithmic merging increased from 30,381,950 pre-finalization writes to 36,141,662 including finalization, compared with 165,003,237 to 170,762,949 for the baseline. Finalizing all disk generations with one k-way merge avoids repeatedly rewriting older postings. Pairwise finalization would reprocess earlier output after each additional generation, increasing I/O despite producing an equivalent final index.

Trade-offs and limitations

Logarithmic merging reduces update-related rewrites at the expense of searching several on-disk indexes. For the four observed configurations, the maximum occupied logarithmic generations were 6, 5, 4 and 3, respectively, compared with a single main disk index for the baseline. The experiments measure index construction, not query latency; therefore no claim about end-to-end query speed follows from these data. Runtime results also depend on cache, storage, compiler, and machine load. The fixed posting-based capacity supports comparable work counts across environments.

Conclusion

The C++ implementation maintained correct dynamic lookups and finalized indexes and reproduced the expected binary-counter behavior of logarithmic merging. Across all measured auxiliary capacities, logarithmic merging reduced total pre-finalization postings written, consistent with its better asymptotic merge cost. The primary trade-off is greater query-time fan-out across occupied generations.

References and data provenance

[1] C. D. Manning, P. Raghavan, H. Schütze, *Introduction to Information Retrieval*, Sections 4.5 and 5.3.1 (2008).
[2] Instructor-provided Logarithmic Merging in C++ assignment and starter framework.
[3] Locally measured results/dynamic.csv, trace_aux_100000.csv, trace_log_100000.csv, and verification logs (October 2026).